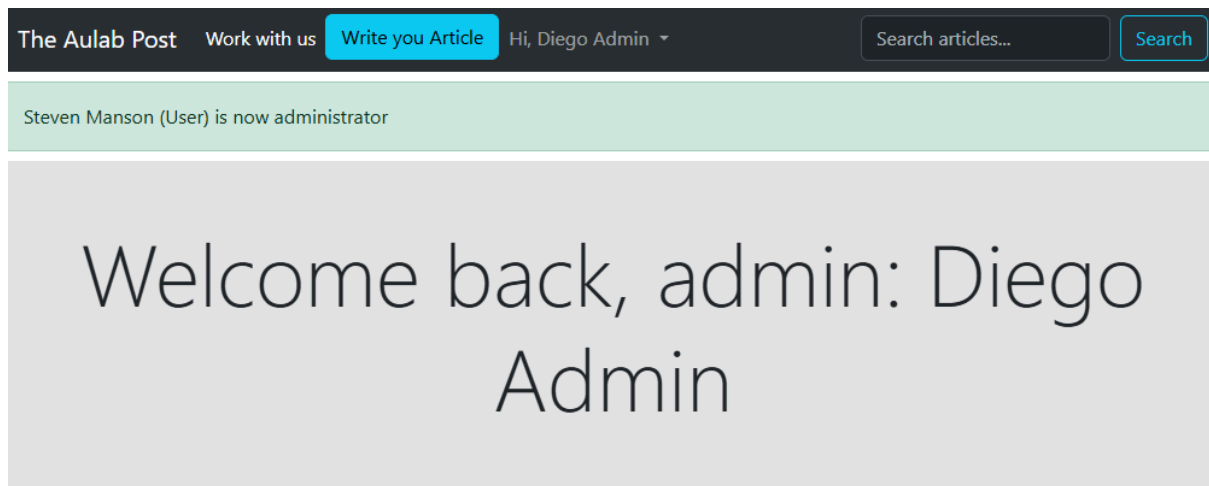


Challenge 2: CSRF su rotte GET

La vulnerabilità CSRF (Cross-Site Request Forgery) è presente perché le rotte di promozione utenti (/admin/{user}/set-admin, ecc.) usano il metodo HTTP GET per modificare dati sensibili.

Un attaccante può creare una pagina HTML malevola che, se aperta da un admin loggato, esegue azioni non autorizzate (es. promuovere un utente a admin).



Mitigazione applicata

Ho convertito le rotte da GET a POST e aggiunto il token CSRF nei form del dashboard.

Ora le rotte accettano solo richieste POST con token valido, bloccando gli attacchi CSRF.

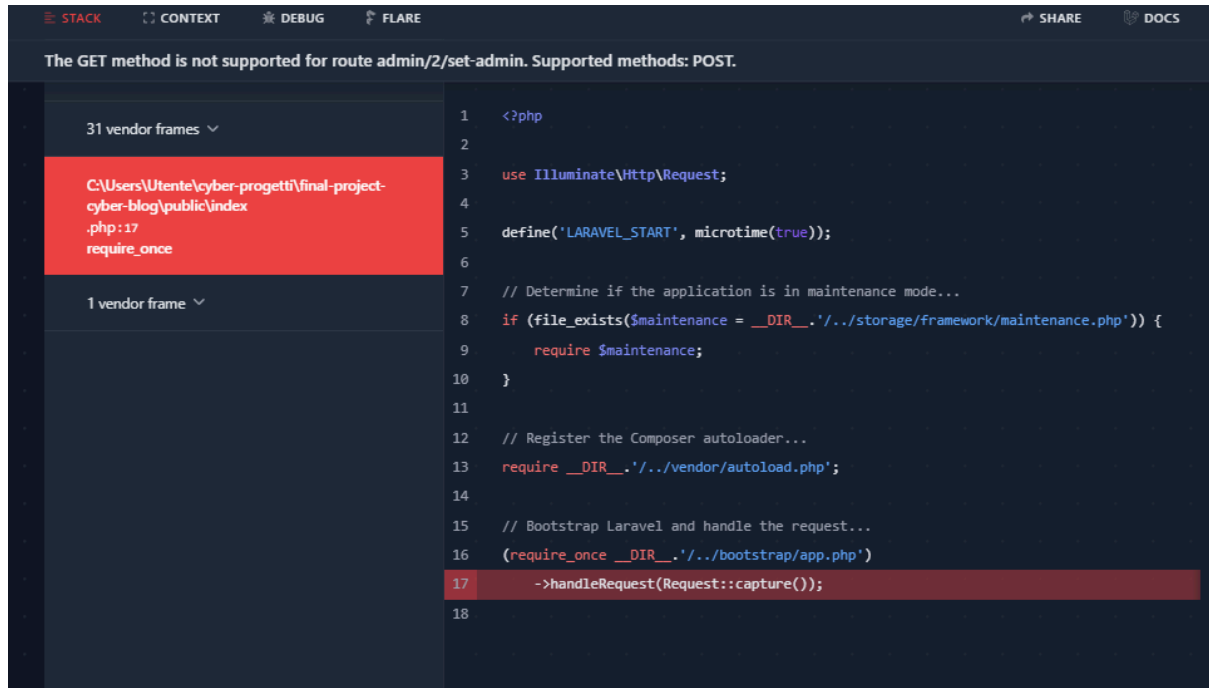
Codice implementato:

In routes/web.php:

```
Route::post('/admin/{user}/set-admin', [AdminController::class,
'setAdmin'])->name('admin.setAdmin');
Route::post('/admin/{user}/set-revisor', [AdminController::class,
'setRevisor'])->name('admin.setRevisor');
Route::post('/admin/{user}/set-writer', [AdminController::class,
'setWriter'])->name('admin.setWriter');
```

In resources/views/components/requests-table.blade.php:

```
<form method="POST" action="{{ route('admin.setAdmin', $user) }}"
style="display:inline;">
    @csrf
    <button type="submit" class="btn btn-secondary">Enable
admin</button>
</form>
```



Conclusion:

La mitigazione è efficace: l'attacco CSRF fallisce con errore 405 Method Not Allowed, proteggendo l'applicazione.